Programación I

Maestría en Ciencia de Datos

Sesión 3

IT PhD Rocío del Carmen Chávez Álvarez

Listas (Continuación de la sesión 2)

```
datos_varios=[3, 5, 7, ”escritorio”,”borrador”, 100]

datos_varios.insert(3,”folder”)

datos_varios

[3, 5, 7, ‘folder’, ‘escritorio’, ’borrador’, 100]
```

Listas (Continuación)

Buscando un elemento en una lista

```
datos_varios=[3, 5, 7, ”escritorio”,”borrador”, 100]

print(”escritorio” in datos_varios)
```

True

```
print(”jirafa” in datos_varios)
```

False

Listas (Continuación)

Eliminando el último elemento en una lista

```
datos_varios=[3, 5, 7, ”escritorio”,”borrador”, 100]

datos_varios.pop()

print(datos_varios[ : ])
```

[3, 5, 7, ”escritorio”,”borrador”]

Listas (Continuación)

Concatenando dos listas

```
datos_varios=[3, 5, 7, ”escritorio”,”borrador”, 100]

datos_varios2=[”mochila”, True]

datos_varios3 = datos_varios + datos_varios2

print(datos_varios3[ : ]) o datos_varios3
```

[3, 5, 7, ”escritorio”,”borrador”, 100, “mochila”, True]

Listas (Continuación)

Repitiendo los elementos de las listas

```
datos_varios=[3, 5, 7, ”escritorio”,”borrador”, 100] * 2

print(datos_varios[ : ])
```

```
[3, 5, 7, ”escritorio”,”borrador”, 100, 3, 5, 7, ”escritorio”,”borrador”, 100]
```

Unidad 3.
Funciones con Cadena

Función Strings

Formatted string literals

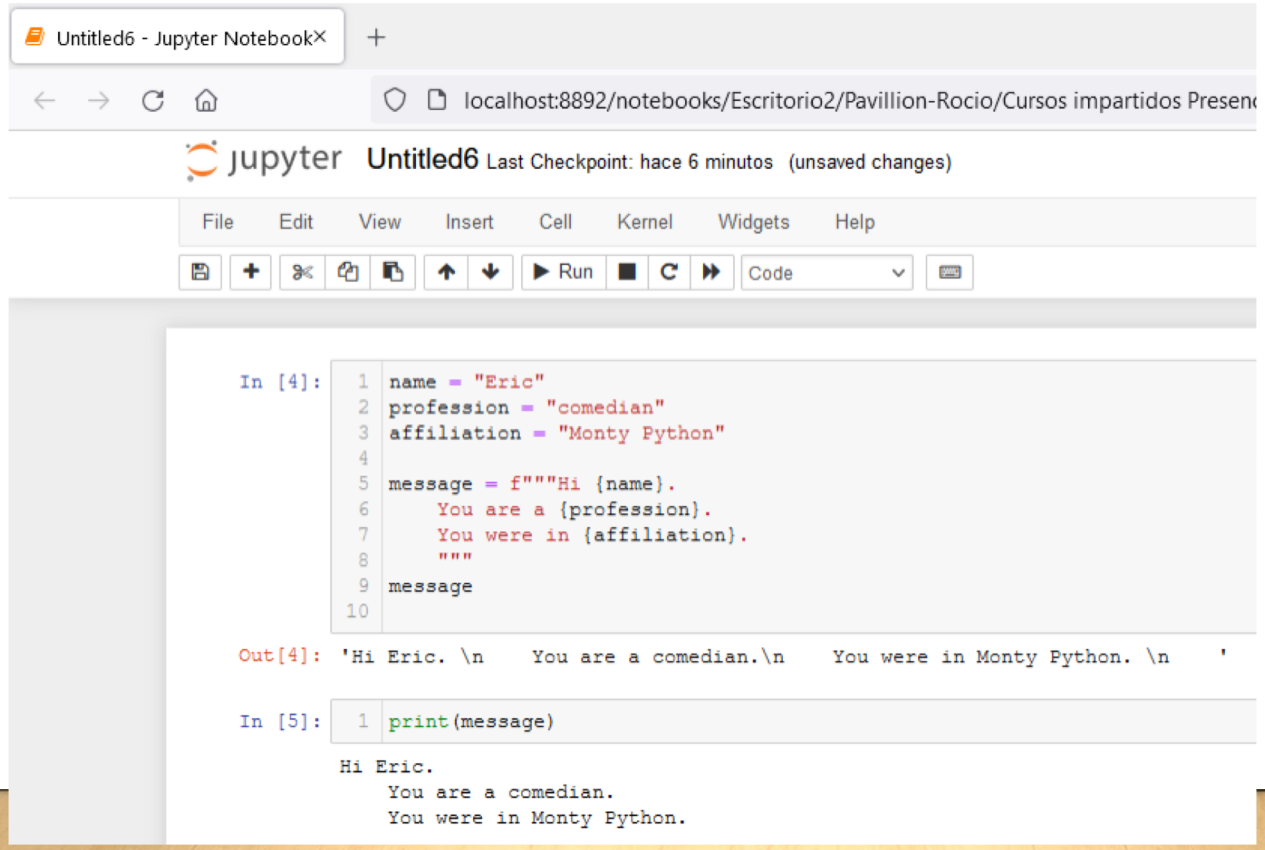
f-strings are string literals that have an **f** or an **F** at the beginning and curly braces containing expressions that will be replaced with their values

```
name = "Eric"
age = 74
print(f"Hello, {name}. You are {age}.“)
'Hello, Eric. You are 74.'

>>> f"{2 * 37}“
'74'
```

f Strings (Cont.)

```
def to_lowercase(input):
        return input.lower()

name = "Eric Idle"
print(f"{to_lowercase(name)} is funny." )
'eric idle is funny.'


print(f"{name.lower()} is funny." )
‘eric idle is funny.'
```

f Strings (Cont.)



Función dir()

Es útil cuando queremos saber qué métodos podemos aplicar sobre una variable dependiendo del tipo de dato del que se trate

```
In [21]:  1 x=3
          2 dir(x)

Out[21]: ['__abs__',
          '__add__',
          '__and__',
          '__bool__',
          '__ceil__',
          '__class__',
          '__delattr__',
          '__dir__',
          '__divmod__',
          '__doc__',
          '__eq__',
          '__float__',
          '__floor__',
          '__floordiv__',
          '__format__',
          '__ge__',
          '__getattribute__',
          '__getnewargs__',
          '__gt__',
          '__hash__',
          '__index__',
          '__init__',
          '__init_subclass__',
          '__int__',
          '__invert__',
          '__le__',
          '__lshift__',
          '__lt__',
          '__mod__',
          '__mul__',
          '__ne__',
          '__neg__',
          '__new__',
          '__or__',
          '__pos__',
          '__pow__',
          '__radd__',
          '__rand__',
          '__rdivmod__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__rfloordiv__',
          '__rlshift__',
          '__rmod__',
          '__rmul__',
          '__ror__',
          '__round__',
          '__rpow__',
          '__rrshift__',
          '__rshift__',
          '__rsub__',
          '__rtruediv__',
          '__rxor__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__sub__',
          '__subclasshook__',
          '__truediv__',
          '__trunc__',
          '__xor__',
          'bit_length',
          'conjugate',
          'denominator',
          'from_bytes',
          'imag',
          'numerator',
          'real',
          'to_bytes']
```

```
In [26]:  1 z="Rocio del Carmen"
          2 dir(z)

Out[26]: ['__add__',
          '__class__',
          '__contains__',
          '__delattr__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattribute__',
          '__getitem__',
          '__getnewargs__',
          '__gt__',
          '__hash__',
          '__init__',
          '__init_subclass__',
          '__iter__',
          '__le__',
          '__len__',
          '__lt__',
          '__mod__',
          '__mul__',
          '__ne__',
          '__new__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__rmod__',
          '__rmul__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__subclasshook__',
          'capitalize',
          'casefold',
          'center',
          'count',
          'encode',
          'endswith',
          'expandtabs',
          'find',
          'format',
          'format_map',
          'index',
          'isalnum',
          'isalpha',
          'isdecimal',
          'isdigit',
          'isidentifier',
          'islower',
          'isnumeric',
          'isprintable',
          'isspace',
          'istitle',
          'isupper',
          'join',
          'ljust',
          'lower',
          'lstrip',
          'maketrans',
          'partition',
          'replace',
          'rfind',
          'rindex',
          'rjust',
          'rpartition',
          'rsplit',
          'rstrip',
          'split',
          'splitlines',
          'startswith',
          'strip',
          'swapcase',
          'title',
          'translate',
          'upper',
          'zfill']
```

Ejemplo de Funciones Aplicadas sobre Strings

find()

rfind()

upper()

lower()

replace()

Funciones find() y rfind()

```
1 a="Parangaricutirimicuaro"
2 b="a"
3
4 print(f"La primer letra {b} en {a}, se encuentra en la posicion: ")
5 print(a.find(b))
6
```

```
La primer letra a en Parangaricutirimicuaro, se encuentra en la posicion:
1
```

```
1 a="Parangaricutirimicuaro"
2 b="i"
3
4 print(f"La primer letra {b} en {a}, se encuentra en la posicion: ")
5 print(a.find(b))
6
```

```
La primer letra i en Parangaricutirimicuaro, se encuentra en la posicion:
8
```

```
1 a="Parangaricutirimicuaro"
2 b="a"
3
4 print(f"La última letra {b} en {a}, se encuentra en la posicion: ")
5 print(a.rfind(b))
6
```

```
La última letra a en Parangaricutirimicuaro, se encuentra en la posicion:
19
```

```
1 a="Parangaricutirimicuaro"
2 b="i"
3
4 print(f"La última letra {b} en {a}, se encuentra en la posicion: ")
5 print(a.rfind(b))
6
```

```
La última letra i en Parangaricutirimicuaro, se encuentra en la posicion:
16
```

Funciones lower() y upper()

```
1  a="Parangaricutirimicuaro"
2
3  print(a.lower())
4
5
```

```
parangaricutirimicuaro
```

```
1  a="Parangaricutirimicuaro"
2
3  print(a.upper())
4
5
```

```
PARANGARICUTIRIMICUARO
```

Función replace()

```
1  a="Tarjeta Aprobada"
2
3  print(a.replace("Aprobada", "Rechazada"))
4
5
```

```
Tarjeta Rechazada
```

Aplicamos upper() a replace()

```
1  a="Tarjeta Aprobada"
2
3  print(a.replace("Aprobada", "Rechazada").upper())
4
5
```

```
TARJETA RECHAZADA
```

Funciones len(), split() y join()

```
1  a="Tarjeta Aprobada"
2
3  print(len(a))
4
5
```

```
16
```

```
1  a="Tarjeta Aprobada"
2
3  print(a.split(" ") )
4
5
```

```
['Tarjeta', 'Aprobada']
```

```
1  a=["Tarjeta", "Aprobada"]
2
3  print(" ".join(a))
4
5
```

```
Tarjeta Aprobada
```

Operadores de Comparación

Operadores de Comparación

== Igual que

!= diferente a

< Menor que

> Mayor que

>= Mayor o igual que

<= Menor o igual que

Unidad 2.
Fundamentos de Programación en Python

2. Creación de Pseudocódigo y de Diagramas de Flujo

Ejercicio1 Comparación de dos Números

Solicitar que se introduzcan dos número por medio del teclado

Mostrar en pantalla:

El primer número que se introdujo

El segundo número que se introdujo

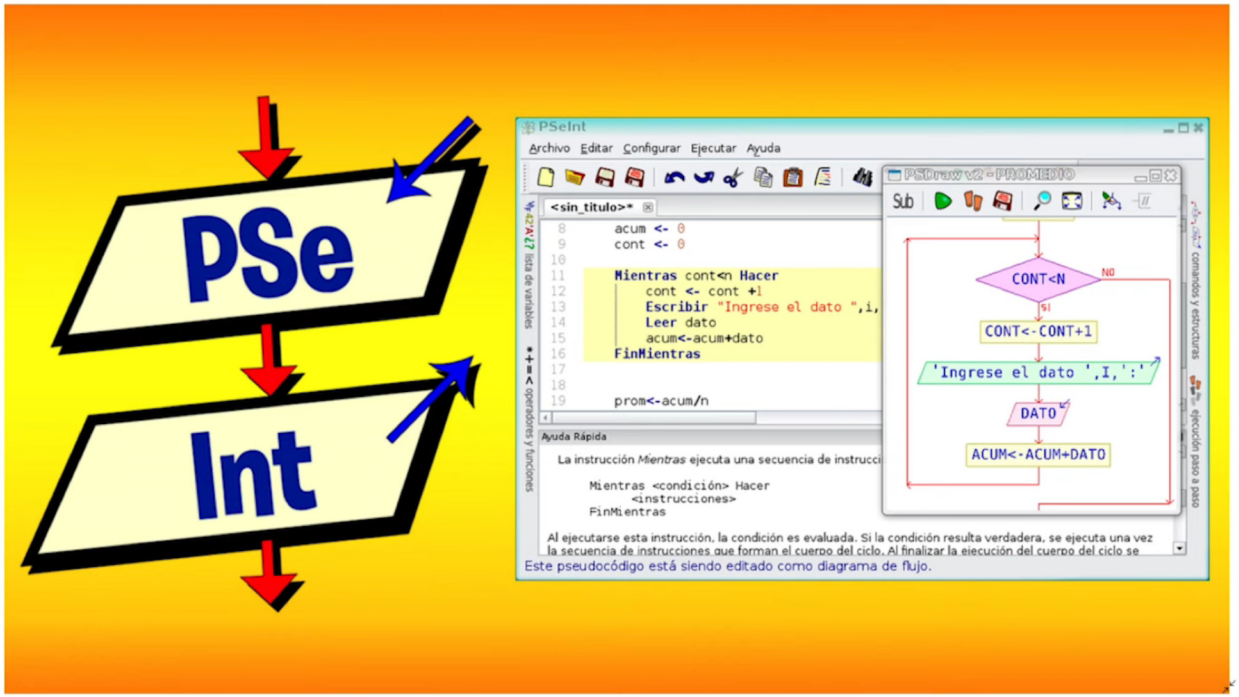
La relación que existe entre ellos:

- Si ambos números son iguales

- Si el primero es menor que el segundo o

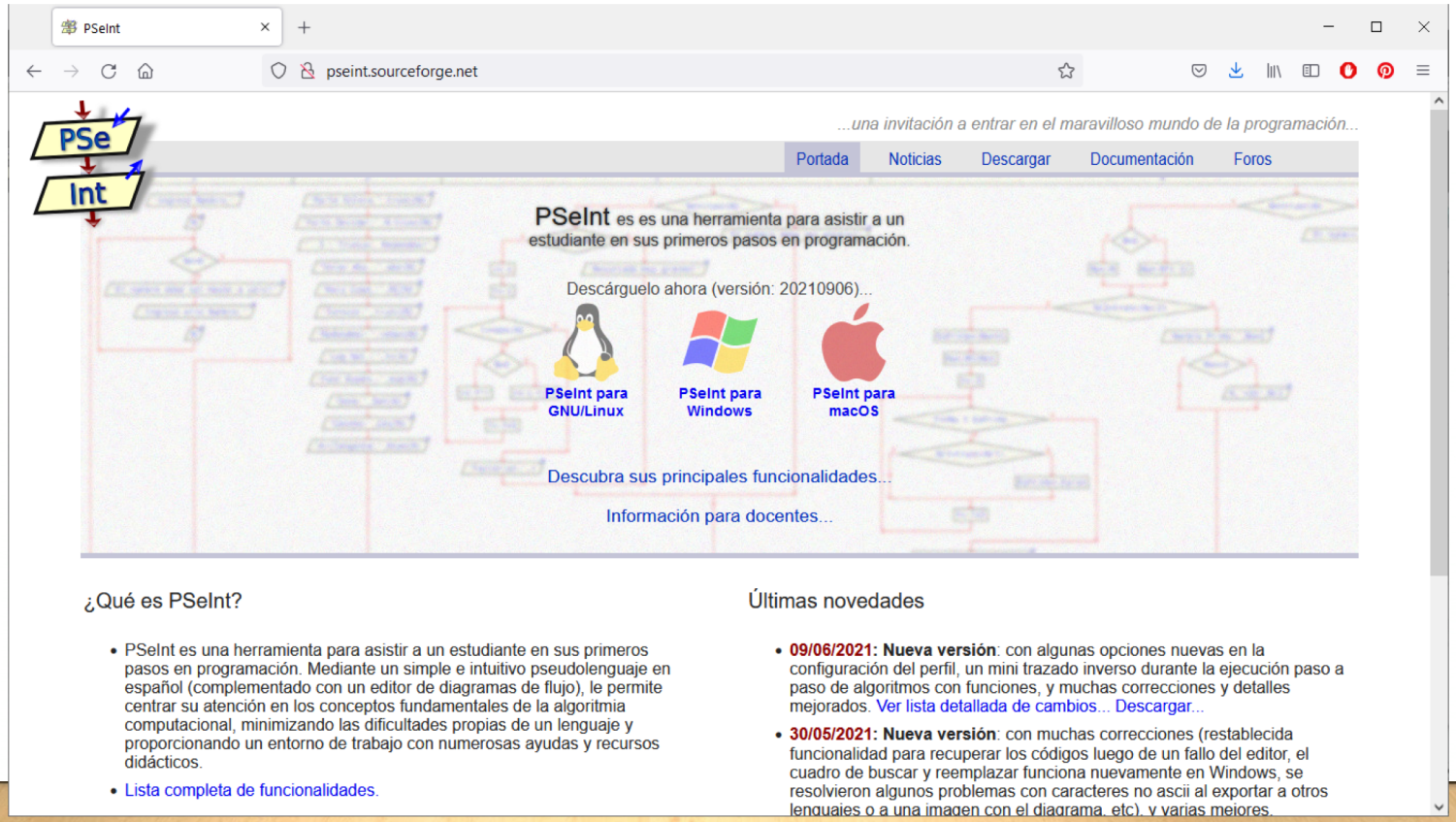
-Si el primero es mayor que el segundo

Pseudocódigo y Diagramas de flujo con PSeInt

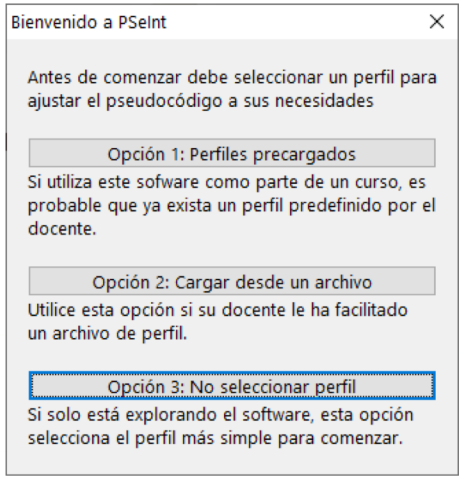


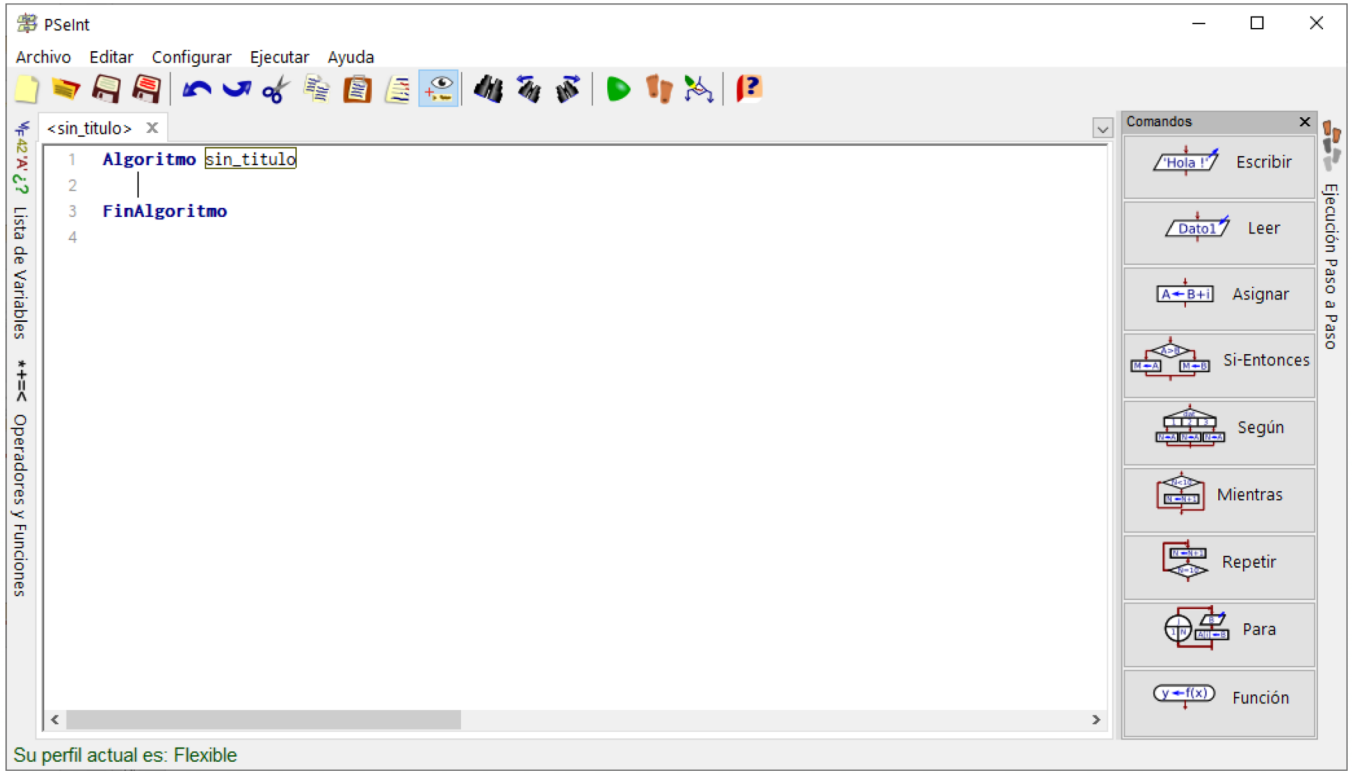
Imágen tomada de: https://www.youtube.com/watch?v=FvibfpSVFBw&ab_channel=Danisable

Descargar PSeInt

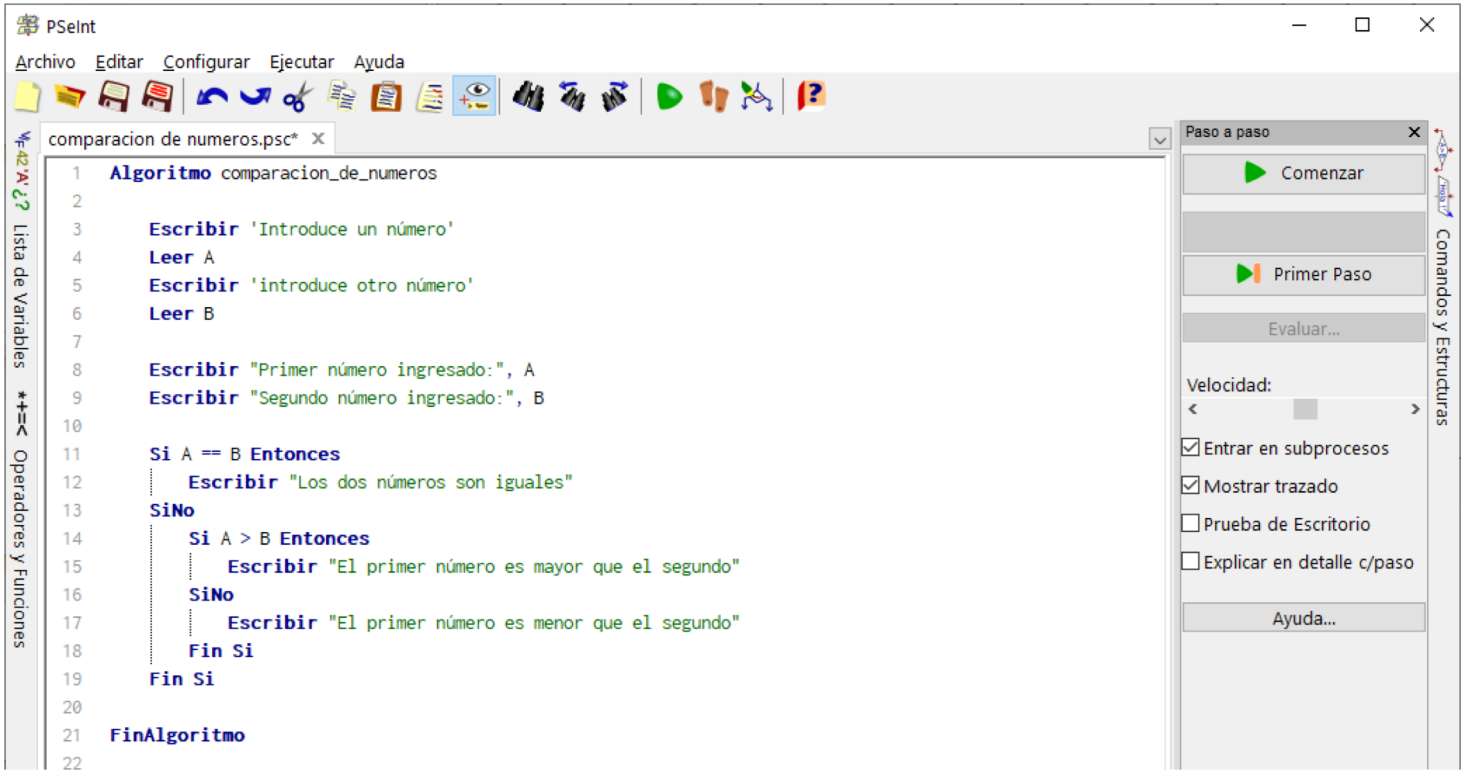


Instalando PSeInt

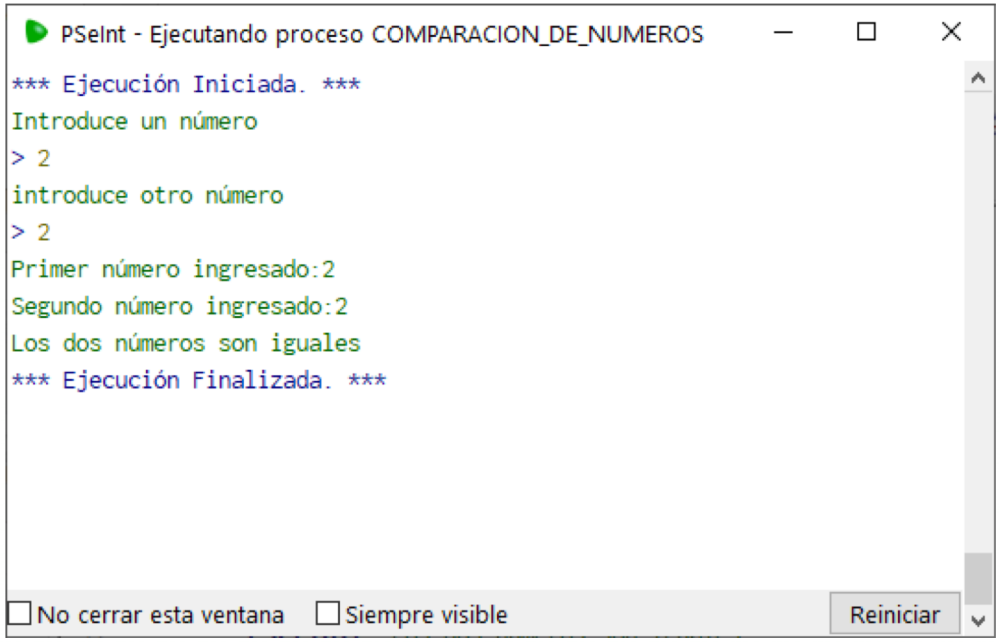




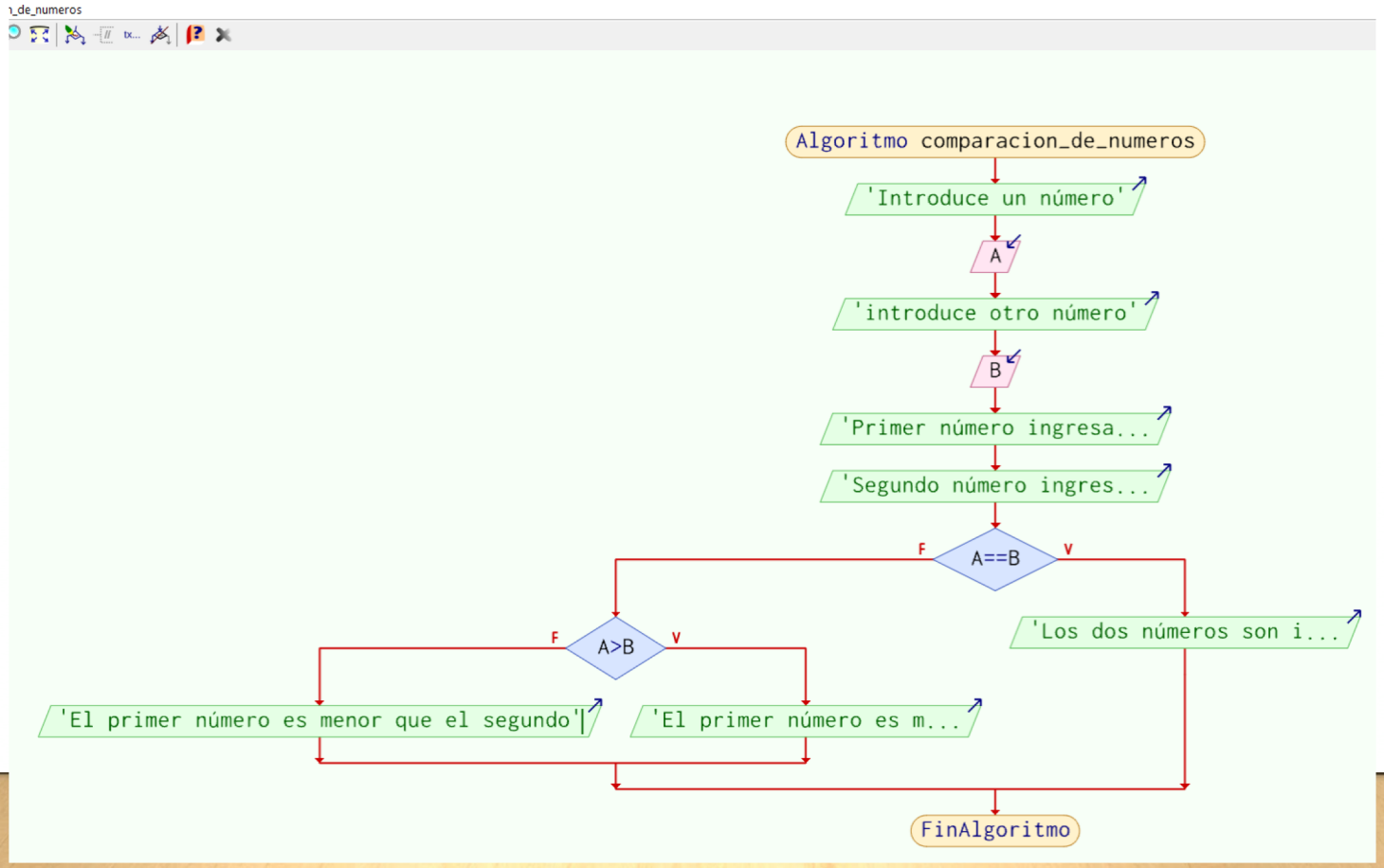
Pseudocódigo en PSeInt



Ejecutando el Pseudocódigo en PSeInt



Obtenemos el diagrama de Flujo



Ejercicio 2 Desplegando la Tasa de Impuesto

Objetivo: Vamos a informar al usuario la tarifa de impuestos que debe pagar

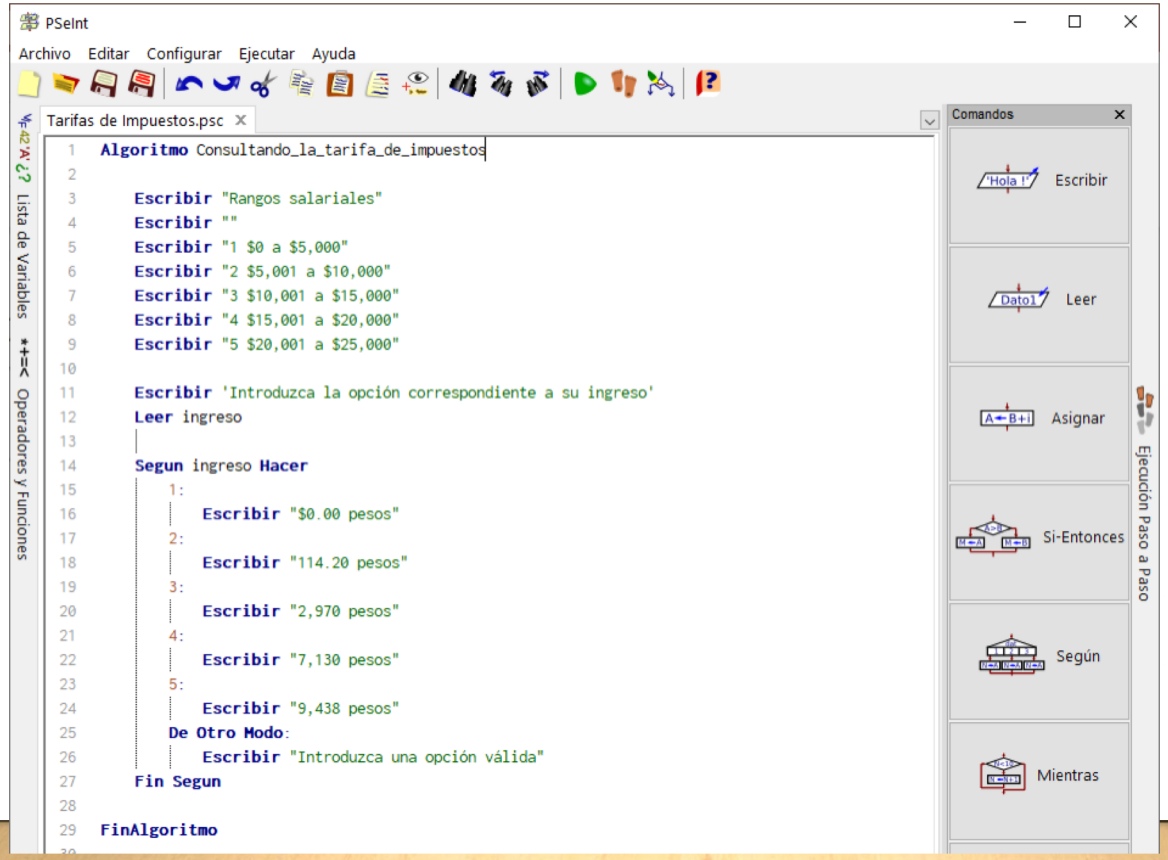
Las tarifas que existen para pagar los impuestos son:

1	$0 a $5,000	"$0.00 pesos"
2	$5,001 a $10,000	"114.20 pesos"
3	$10,001 a $15,000	"2,970 pesos"
4	$15,001 a $20,000	"7,130 pesos"
5	$20,001 a $25,000	"9,438 pesos"

Pedir que introduzca la opción a la que pertenece su ingreso

Desplegar la cuota a pagar con base en el ingreso que percibe

Pseudocódigo del Ejercicio 2 en PSeInt




```
Algoritmo Consultando_la_tarifa_de_impuestos

    Escribir "Rangos salariales"
    Escribir ""
    Escribir "1 $0 a $5,000"
    Escribir "2 $5,001 a $10,000"
    Escribir "3 $10,001 a $15,000"
    Escribir "4 $15,001 a $20,000"
    Escribir "5 $20,001 a $25,000"

    Escribir 'Introduzca la opción correspondiente a su ingreso'
    Leer ingreso

    Segun ingreso Hacer
        1:
            Escribir "$0.00 pesos"
        2:
            Escribir "114.20 pesos"
        3:
            Escribir "2,970 pesos"
        4:
            Escribir "7,130 pesos"
        5:
            Escribir "9,438 pesos"
        De Otro Modo:
            Escribir "Introduzca una opción válida"
    Fin Segun

FinAlgoritmo
```

Ejecutando el Pseudocódigo

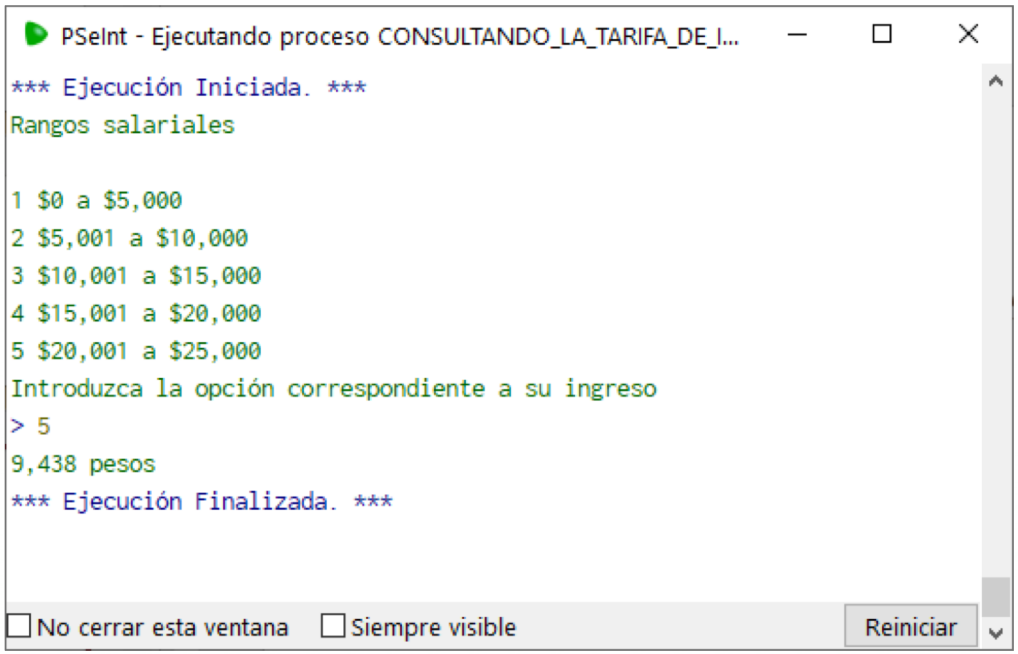
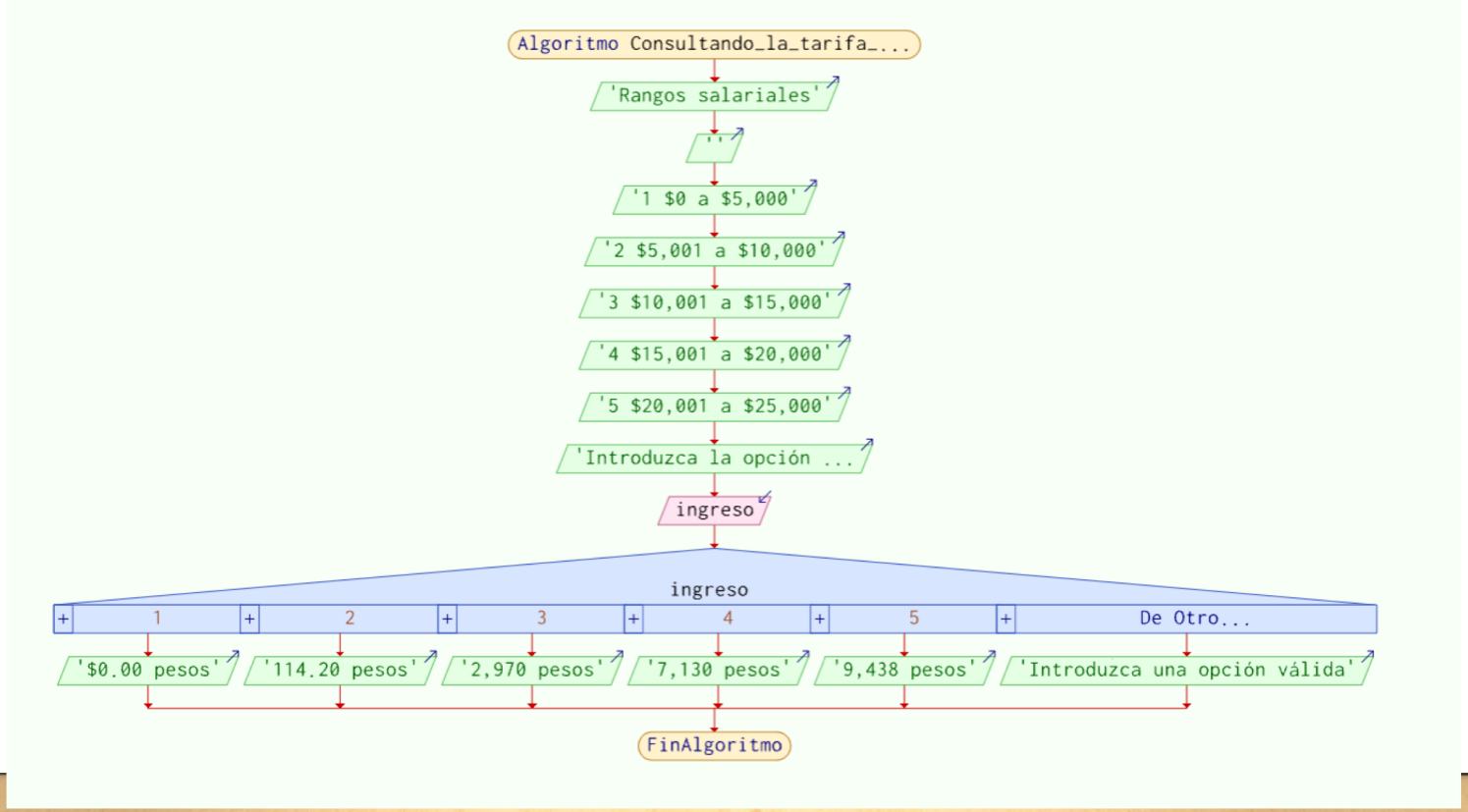


Diagrama de Flujo



Ejercicio 2 Solución Propuesta

File Edit View Insert Cell Kernel Widgets Help

Run Code

In [12]:
```
print("Rangos salariales: \n 1 $0 a $5,000 \n 2 $5,001 a $10,000 \n 3 $10,001 a $15,000 \n 4 $
```

```
Rangos salariales:
 1 $0 a $5,000
 2 $5,001 a $10,000
 3 $10,001 a $15,000
 4 $15,001 a $20,000
 5 $20,001 a $25,000
```

In [17]:
```
ingreso=int(input('Introduzca la opción correspondiente a su ingreso\n'))
```

```
Introduzca la opción correspondiente a su ingreso
6
```

In [19]:
```
if ingreso == 1:
    print("$0.00 pesos")
elif ingreso == 2:
    print("114.20 pesos")
elif ingreso == 3:
    print("2,970 pesos")
elif ingreso == 4:
    print("7,130 pesos")
elif ingreso == 5:
    print("9,438 pesos")
else:
    print("Introduzca una opción válida")

```

```
Introduzca una opción válida
```

Ejemplos de Pseudocódigos en Internet

http://www.discoduroderoer.es/ejercicios-propuestos-y-resueltos-en-pseudocodigo/#

Funciones

¿Qué son las funciones?

- Bloques de código que funcionan como una unidad para llevar a cabo una tarea
- Pueden devolver valores o no
- Pueden tener parámetros o no
- Cuando se encuentran definidas dentro de una clase, las funciones también denominan métodos
- Permiten reutilizar el código en caso de ser necesario

Sintaxis de las Funciones

```
def nombre_función():
    instrucciones de la función
    return (opcional)
```

```
def nombre_función(parámetros):
    instrucciones de la función
    return (opcional)
```

Ejecución de una Función

nombre_función()

nombre_función(parámetros)

Sintaxis de las Funciones

```
def nombre_función():
    instrucciones de la función
    return (opcional)
```

Declaración de la función

Cuerpo de la función

```
nombre_función()
```

Llamada a la función

Ejemplos de Funciones

```
1 def saludo():
2     print("Hola")
```

No recibe parámetros y no retorna un resultado

```
1 saludo()
```

```
Hola
```

Recibe parámetros y no retorna un resultado

```
1 def bienvenida(curso):
2   print("Bienvenidos al curso" + curso + "!")
```

```
1 bienvenida("Programación I")
```

```
Bienvenidos al cursoPrograмación I!
```

```
1 def suma(num1, num2):
2     resultado = num1 + num2
3     return resultado
4
```

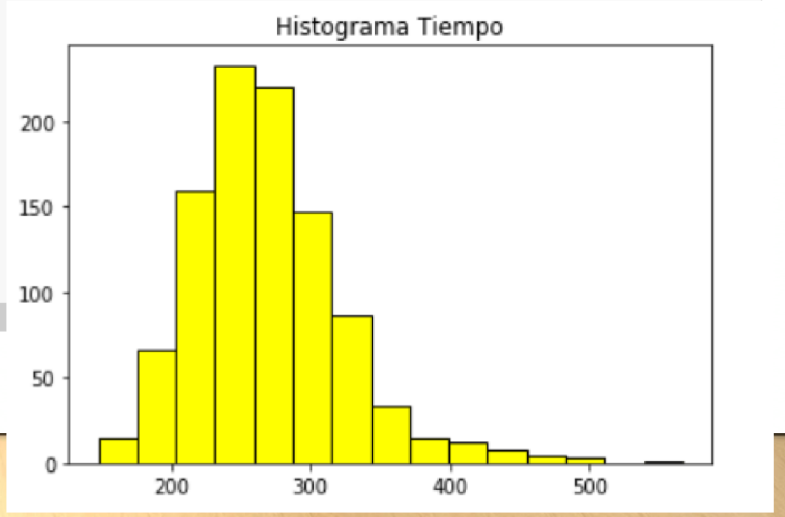
Recibe parámetros y retorna un resultado

```
1 suma(3,5)
```

```
8
```

Histograma sin funciones

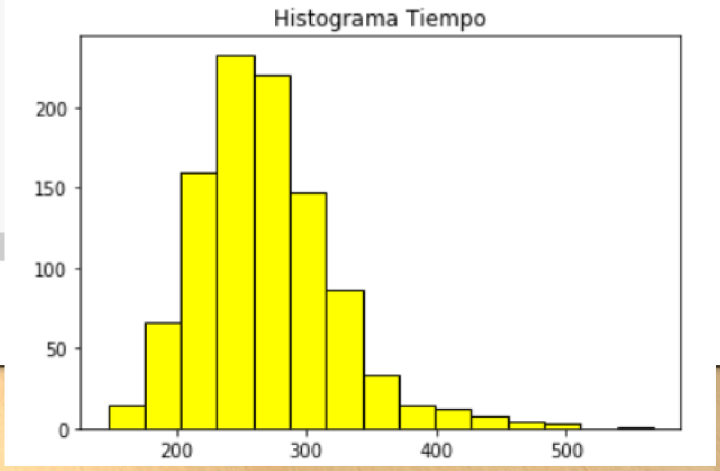
```
#Indicamos el directorio en el que vamos a trabajar
import os
os.chdir('C:/Users/rocio/Escritorio2/Pavillion-Rocio/Cursos impartidos Presenciales/Programacion I MCD/Bases de Dato

#Importamos los paquetes que vamos a necesitar
import pandas as pd #Contiene funciones que nos ayudan en el análisis de datos
import matplotlib.pyplot as plt #Nota: Fue necesario instalar matplotlib 3.4.3 con conda install matplotlib=3.4.3
                                #y comentar la linea 4 del archivo _classic_test_patch.mplstyle

#Leemos el archivo a analizar
atletas = pd.read_csv('categorias de corredores.csv', index_col=0) #Con index_col=0 le indicamos que las filas tiene

#Vemos las primeras filas del archivo
atletas.head()

#Creamos el histograma de la variable Tiempo
plt.figure(1)
plt.hist(atletas['Tiempo'], 15, color="yellow", ec="black")
plt.title("Histograma Tiempo")
plt.savefig("Histograma.jpg")
```



Histograma con funciones

```python
#Indicamos el directorio en el que vamos a trabajar
import os
os.chdir('C:/Users/rocio/Escritorio2/Pavillion-Rocio/Cursos impartidos Presenciales/Programacion I MCD/Bases de Datc

#Importamos los paquetes que vamos a necesitar
import pandas as pd #Contiene funciones que nos ayudan en el análisis de datos
import matplotlib.pyplot as plt

#Leemos el archivo a analizar
atletas = pd.read_csv('categorias de corredores.csv', index_col=0) #Con i

#Creamos una función que cree el histograma
def crear_histograma(variable):
    plt.figure(1)
    plt.hist(atletas[variable], 15, color="yellow", ec="black")
    plt.title("Histograma Tiempo")
    plt.savefig("Histograma.jpg")

crear_histograma("Tiempo")
```

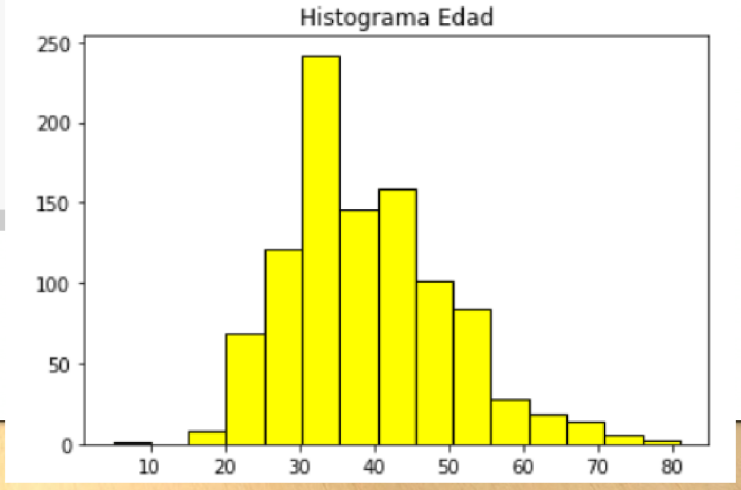
```python

crear_histograma("Tiempo")
```



Histograma con funciones (Cont.)

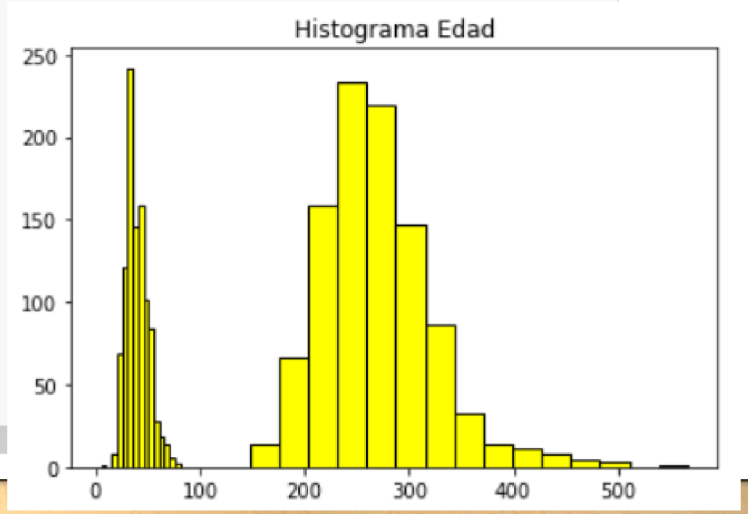
```python
#Indicamos el directorio en el que vamos a trabajar
import os
os.chdir('C:/Users/rocio/Escritorio2/Pavillion-Rocio/Cursos impartidos Presenciales/Programacion I MCD/Bases de Datc

#Importamos los paquetes que vamos a necesitar
import pandas as pd #Contiene funciones que nos ayudan en el análisis de datos
import matplotlib.pyplot as plt

#Leemos el archivo a analizar
atletas = pd.read_csv('categorias de corredores.csv', index_col=0) #Co
atletas.head()

#Creamos una función que cree el histograma
def crear_histograma(variable):
    plt.figure(1)
    plt.hist(atletas[variable], 15, color="yellow", ec="black")
    plt.title(f"Histograma {variable}")
    plt.savefig(f"{variable}.jpg")

```

```python

crear_histograma("Edad")
```



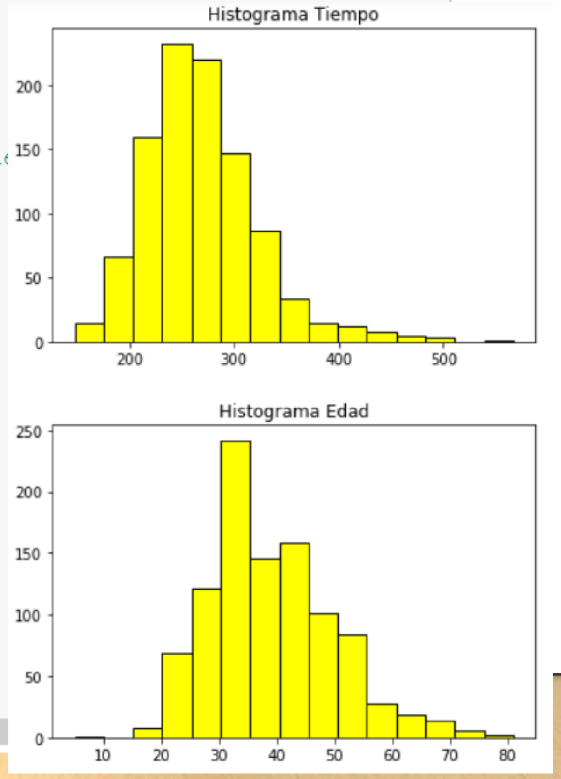
Dos Histogramas con una Función

```python
#Indicamos el directorio en el que vamos a trabajar
import os
os.chdir('C:/Users/rocio/Escritorio2/Pavillion-Rocio/Cursos impartidos Presenciales/Programacion I MCD/Bases de Datc

#Importamos los paquetes que vamos a necesitar
import pandas as pd #Contiene funciones que nos ayudan en el análisis de datos
import matplotlib.pyplot as plt

#Leemos el archivo a analizar
atletas = pd.read_csv('categorias de corredores.csv', index_col=0) #Con index_col=0 le indicamos que las filas tiene
atletas.head()

#Creamos una función que cree el histograma
def crear_histograma(variable):
    plt.hist(atletas[variable], 15, color="yellow", ec="black")
    plt.title(f"Histograma {variable}")
    plt.savefig(f"{variable}.jpg")


variable=["Tiempo", "Edad"]

for i in variable:
    crear_histograma(i)

```



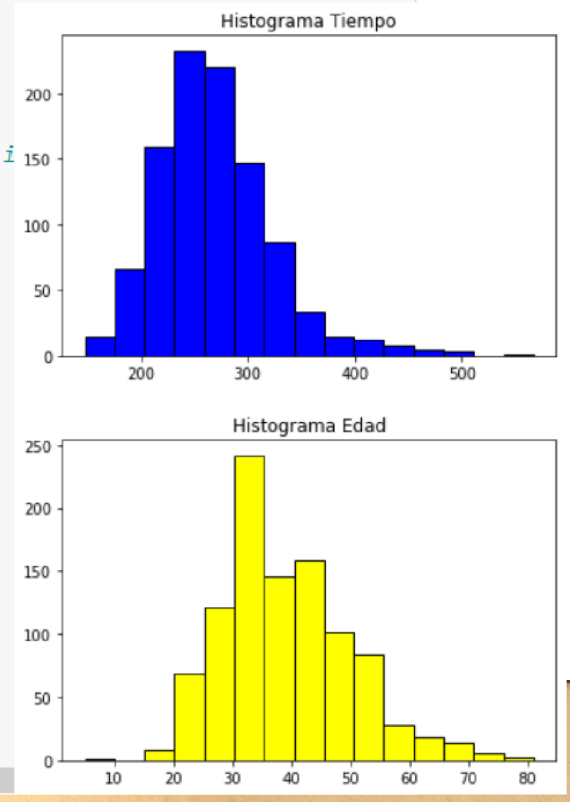
Desplegando los dos Histogramas Correctamente

```python
#Indicamos el directorio en el que vamos a trabajar
import os
os.chdir('C:/Users/rocio/Escritorio2/Pavillion-Rocio/Cursos impartidos Presenciales/Programacion I MCD/Bases de Dato

#Importamos los paquetes que vamos a necesitar
import pandas as pd #Contiene funciones que nos ayudan en el análisis de datos
import matplotlib.pyplot as plt

#Leemos el archivo a analizar
atletas = pd.read_csv('categorias de corredores.csv', index_col=0) #Con index_col=0 le
atletas.head()

#Creamos una función que cree el histograma
def crear_histograma(variable,j):
    plt.figure(j)
    plt.hist(atletas[variable], 15, color="yellow", ec="black")
    plt.title(f"Histograma {variable}")
    plt.savefig(f"{variable}.jpg")


variable=["Tiempo", "Edad"]
j=1

for i in variable:
    crear_histograma(i,j)
    j +=1
```



Utilizando diferentes colores en los Histogramas

```
#Indicamos el directorio en el que vamos a trabajar
import os
os.chdir('C:/Users/rocio/Escritorio2/Pavillion-Rocio/Cursos impartidos Presenciales/Programacion I MCD/Bases de Datc

#Importamos los paquetes que vamos a necesitar
import pandas as pd #Contiene funciones que nos ayudan en el análisis de datos
import matplotlib.pyplot as plt

#Leemos el archivo a analizar
atletas = pd.read_csv('categorias de corredores.csv', index_col=0) #Con index_col=0 le i
atletas.head()

#Creamos una función que cree el histograma
def crear_histograma(variable,j,z):
    plt.figure(j)
    plt.hist(atletas[variable], 15, color= z, ec="black")
    plt.title(f"Histograma {variable}")
    plt.savefig(f"{variable}.jpg")


variable=["Tiempo", "Edad"]
colores=["blue", "yellow"]
j=1
z=0

for i  in variable:
    crear_histograma(i,j,colores[z])
    j +=1
    z +=1

```



Tarea 3

Copiar los ejemplos que hicimos el día de hoy a su formulario

Subir el script al classroom

Generar:

- Pseudocódigo
- Diagrama de flujo
- Script

Que lleve a cabo lo siguiente:….

Modificar el archivo con el que creamos los histogramas con funciones, de manera que ahora nos genere gráficos de barras.

Cada gráfico de barras deberá tener un color diferente